

SECTION 31 — DESIGN PATTERNS FOR TEST AUTOMATION (Refined)

Topics Covered

- 31.1-31.14 (14 headers)

First pass — 2 parallel batch subagents

31.1 Factory Pattern — Refined

Theory: Simple Factory centralizes object creation behind a static method, hiding `new` logic from test code. For SDET work: create `WebDriver`, `APIClient`, or `PageObject` instances based on a string/enum — switch logic in one place. Reduces duplication, eases browser/environment switching. Caveat: overgrown switch violates Open-Closed (add browser = modify factory); prefer enums over raw strings for type safety. Enterprise Relevance: Factory is the extension point for cross-browser/env matrices — new browser = 1 case, zero test changes. Enum keys fail at compile time; string keys fail at runtime in CI.

```
public class DriverFactory {
    public static WebDriver create(Browser browser) {
        return switch (browser) {
            case CHROME -> new ChromeDriver();
            case FIREFOX -> new FirefoxDriver();
            default -> throw new IllegalArgumentException(browser.name());
        };
    }
}
```

Triage: unsupported-browser at runtime → stringly config; switch sprawl → Factory Method/DI. Anti: `new ChromeDriver()` in tests; string keys; 50-case switch (split by family). Qs: Simple Factory vs Factory Method? When does the switch become a smell?

31.2 Factory Method — Refined

Theory: Factory Method defines an interface for creation but lets subclasses decide the concrete type — delegating instantiation via overridden methods. Unlike Simple Factory's switch, it uses polymorphism: `TestBase.createDriver()` is abstract; `ChromeTest` / `FirefoxTest` override it. Ideal for cross-browser suites needing extensibility without modifying existing code (Open-Closed) and Playwright/Selenium grid setups. Enterprise Relevance: New browser = new subclass, zero edits to existing code. The OCP-compliant answer when Simple Factory's switch grows unwieldy.

```
abstract class TestBase {
    abstract WebDriver createDriver();
    void setup() { WebDriver d = createDriver(); d.get("https://app.com"); }
}
class ChromeTest extends TestBase {
```

```
@Override WebDriver createDriver() { return new ChromeDriver(); }
}
```

Triage: switch sprawl → migrate to Factory Method/DI; subclass explosion (20 browsers × 3 envs) → parameterized factory instead. Anti: subclass per trivial variant; abstract creator without shared workflow. Qs: Simple Factory vs Factory Method vs Abstract Factory?

31.3 Builder Pattern — Refined

Theory: Builder constructs complex objects step-by-step with fluent chaining, avoiding telescoping constructors (5+ params, ambiguous order). Essential for test data: `User`, `Order`, REST payloads with many optional fields. Improves readability (`new UserBuilder().withAge(30).withRole("admin").build()`), self-documents tests, supports immutable objects. Validate in `build()` to fail fast on missing required fields before API/UI execution. Enterprise Relevance: Builders make test data intent explicit (only overridden fields shown) and immutable (no post-hoc mutation surprises). Required-field validation in `build()` catches bad data at construction, not mid-test.

```
public class User {
    String name, role;
    public static class Builder {
        User u = new User();
        public Builder name(String n) { u.name = n; return this; }
        public Builder role(String r) { u.role = r; return this; }
        public User build() { Objects.requireNonNull(u.name, "name required"); return u; }
    }
}
// User admin = new User.Builder().name("a").role("ADMIN").build();
```

Triage: telescoping constructors (which String is which?) → builder; invalid data mid-test → build-time validation. Anti: setters instead of builder (mutable); no validation (bad data travels); builder performing I/O. Qs: Builder vs telescoping vs setters?

31.4 Strategy Pattern — Refined

Theory: Strategy encapsulates interchangeable algorithms behind a common interface, injected at runtime. Use for varied validations, retry policies, pricing rules, locator strategies — without `if-else` chains. In automation: swap `PaymentStrategy` (Card vs UPI) or `WaitStrategy` per test. Boosts maintainability and unit-testability via mocks. Avoid overuse for only two trivial branches (simple conditional suffices). Enterprise Relevance: Strategy turns environment/variant branching (payment methods, retry policies per suite, browser-specific waits) into injectable policies — tested in isolation, composed per need.

```
interface PayStrategy { void pay(int amt); }
class CardPay implements PayStrategy {
    public void pay(int amt) { System.out.println("Card: " + amt); }
}
class Checkout {
    private final PayStrategy s; Checkout(PayStrategy s) { this.s = s; }
```

```
void complete(int a) { s.pay(a); }
}
```

Triage: if-else sprawl on type codes → strategy map; two branches only → keep conditional (YAGNI).
 Anti: strategy per trivial case (over-engineering); stateful strategies shared across tests. Qs: Strategy vs if-else vs polymorphism — when each?

31.5 Singleton — Risks and Appropriate Use — Refined

Theory: Singleton guarantees one shared instance via private constructor + `getInstance()` — useful for `ConfigReader`, `ExtentReports`, `DriverManager` (avoid resource duplication). Risks: hidden global state causes test-order dependence + parallel-execution flakiness; thread-safety bugs if lazily initialized naively (double-checked locking needs `volatile`). Mitigate with `volatile` + DCL, enum singleton (simplest thread-safe), or better: dependency injection for isolated tests. Enterprise Relevance: Singletons are convenient but test-hostile (global state breaks isolation). Use for truly singular expensive resources (config, report); prefer DI/factories for drivers (parallel needs isolation, Sec 22.12).

```
public class Config {
    private static volatile Config instance;
    private Config() {}
    public static Config getInstance() {
        if (instance == null) synchronized (Config.class) {
            if (instance == null) instance = new Config();
        }
        return instance;
    }
}
// simplest: public enum Config { INSTANCE; /* methods */ }
```

Triage: order-dependent failures → singleton state leak; parallel flake → shared singleton mutated. Anti: singleton drivers (parallel clash); lazy without `volatile` (half-constructed publish); singletons for test data. Qs: Enum vs DCL singleton? When is singleton the wrong choice?

31.6 Facade Pattern — Refined

Theory: Facade provides a simplified high-level API over complex subsystems, hiding multi-step workflows. Perfect for SDET helpers: `LoginFacade.login(user)` internally handles navigation, waits, credential entry, OTP. Shortens step definitions, centralizes UI changes to one class, makes BDD/Cucumber layers readable. Avoid God-class trap: facade delegates, never contains business logic or assertions. Enterprise Relevance: Facades are what BDD steps call — one-line steps (`Given logged in as admin`) hiding 10 UI actions. UI flow changes touch 1 facade, not 50 step definitions.

```
public class LoginFacade {
    private final LoginPage lp;
    public LoginFacade(WebDriver d) { lp = new LoginPage(d); }
    public HomePage login(String u, String p) {
        lp.open(); lp.enterUser(u); lp.enterPass(p); return lp.submit();
    }
}
```

```
}  
}
```

Triage: step definitions bloated with UI calls → extract facade; facade with asserts → move to tests.
Anti: God facade (all flows, 2000 lines); business logic in facade; assertions in facade (except navigation checks). Qs: Facade vs Page Object vs Step Definition — what lives where?

31.7 Adapter Pattern — Refined

Theory: Adapter bridges incompatible interfaces without changing existing code — wrapping an adaptee to match a target contract. In test automation: adapt third-party libraries, legacy `OldPaymentGateway`, or varied JSON/XML APIs to a uniform `PaymentProcessor` used by tests. Enables reuse and mocking; supports integration of vendor tools. Adds indirection — document clearly so new SDETs trace delegation during debugging. Enterprise Relevance: Vendor/legacy integration without adapters means test code coupled to third-party APIs (breaks on vendor upgrades). Adapters isolate the coupling to one class.

```
interface Processor { void process(int amt); }  
class LegacyGateway { void charge(double d) { /* legacy */ } }  
class GatewayAdapter implements Processor {  
    private final LegacyGateway g = new LegacyGateway();  
    @Override public void process(int amt) { g.charge((double) amt); }  
}  
// tests depend on Processor; swap LegacyGateway without touching tests
```

Triage: vendor upgrade breaks 50 tests → missing adapter (direct coupling); add adapter + mock for unit speed. Anti: adapters everywhere (indirection fog); leaking adaptee types through adapter. Qs: Adapter vs Facade — both wrap, what differs?

31.8 Dependency Injection — Concept — Refined

Theory: Dependency Injection (DI) is an Inversion of Control technique where a class receives dependencies from outside instead of creating them with `new`. Decouples construction from use — improving testability (inject mocks), reusability, parallel execution (inject per-thread instances). In SDET frameworks: WebDriver, pages, config, clients injected via constructor, Guice/Spring/PicoContainer, or TestNG `@Guice`. Prefer constructor injection (explicit dependencies, single responsibility). Enterprise Relevance: DI makes tests hermetic (mocked deps) and parallel-safe (per-thread instances) by construction. Frameworks without DI hide `new` calls that resist testing.

```
public class LoginPage {  
    private final WebDriver driver;  
    @Inject public LoginPage(WebDriver driver) { this.driver = driver; }  
    public void login(String u, String p) { /* use driver, never create it */ }  
}  
// test: new LoginPage(mockDriver) — no browser needed for logic tests
```

Triage: untestable class (creates own driver/HTTP client inside) → ctor injection; parallel-shared deps → per-thread providers. Anti: field injection (hidden deps); service locator (global lookup); `new` in business logic. Qs: Constructor vs field injection? How does DI enable parallel safety?

31.9 Dependency Inversion — Refined

Theory: Dependency Inversion Principle (DIP — the D in SOLID): high-level modules should depend on abstractions, not low-level details; both should depend on interfaces. For automation: tests depend on `WebDriver`, `UserRepository`, `NotificationSender` interfaces — not `ChromeDriver` or `MySQL` classes. Implementations wired at composition root (DI container, factory). Enables mocking, browser swapping, stable Page Objects without logic changes. Enterprise Relevance: DIP is what lets suites run against mocks (fast PR), staging (nightly), and prod-like (pre-release) by swapping implementations — same tests, different wiring.

```
public interface CustomerDao { Optional<Customer> findById(int id); }
public class CustomerService {
    private final CustomerDao dao;
    public CustomerService(CustomerDao dao) { this.dao = dao; } // depends on
    abstraction
}
// test: new CustomerService(mockDao); prod: new CustomerService(mysqlDao);
```

Triage: unmockable test (concrete `new ChromeDriver` inside) → depend on `WebDriver` + inject; env-specific branches in tests → implementations per env. Anti: depending on concretions; abstractions mirroring one impl (leaky); DI container for 3 classes (overkill). Qs: DI vs DIP — container vs principle? How does DIP enable test doubles?

31.10 Patterns in Selenium — Refined

Theory: Standard Selenium patterns: Page Object Model (encapsulation of locators/actions); Factory for `WebDriver` creation per browser/grid; Singleton for `ConfigReader/DriverManager` (with caveats, Sec 31.5); Strategy for wait/retry or browser selection; `LoadableComponent` (`load()/isLoading()`) and Bot/Fluent patterns for readability + synchronization. Rule: never expose `WebElement` publicly — expose business methods like `loginAs()` returning next page. Enterprise Relevance: This is the Deloitte Selenium blueprint — POM + Factory + ThreadLocal + `LoadableComponent` + Bot actions. Deviations need justification in review.

```
public class DriverFactory {
    public static WebDriver create(String browser) {
        return browser.equals("firefox") ? new FirefoxDriver() : new ChromeDriver();
    }
}
// + LoadableComponent: load() navigates, isLoading() asserts data-test ready-signal
```

Triage: flaky page transitions → `LoadableComponent` `isLoading` guards; public `WebElements` → encapsulate to business methods. Anti: procedural scripts (no POM); static driver; PageFactory proxies (Sec 15.2). Qs: Which 4 patterns form the standard stack? What does `LoadableComponent` add over POM?

31.11 Patterns in REST Assured — Refined

Theory: REST Assured frameworks use Builder for payloads and `RequestSpecBuilder / ResponseSpecBuilder` reuse; Factory for centralized `RequestSpecification` per environment/auth; Singleton for `TokenManager /ExtentReports/Config`; Service Object Model

mirroring POM for endpoints (one client class per resource); Filters for logging/retry; POJOs with Jackson for serialization, schema validation, chaining. Enterprise Relevance: Service Objects are the API POM — endpoint changes touch 1 client. Spec builders + filters standardize auth/logging/timeouts across 500 tests.

```
RequestSpecification spec = new RequestSpecBuilder()
    .setBaseUri(Config.baseUri())
    .addHeader("Authorization", "Bearer " + TokenManager.getToken())
    .setContentType(ContentType.JSON).build();
Response res = given().spec(spec).body(user).when().post("/users");
```

Triage: auth drift → TokenManager central; contract drift → Service Object + schema per endpoint. Anti: inline specs per test; Map payloads (untyped); no schema validation. Qs: Service Object Model — the API equivalent of what Selenium pattern?

31.12 Patterns in Playwright — Refined

Theory: Playwright Java favours POM with composition over inheritance; Fixture/Factory for isolated BrowserContext / Page per test; Builder/Factory for test-data and API setup. Use Playwright.create(), one context per test for parallel safety; components for header/nav fragments; expect(page.locator()) auto-wait assertions. Inject Page via constructor; avoid static shared Page and thread-unsafe singletons. Enterprise Relevance: Playwright patterns center on isolation (context-per-test) + composition (components) + fixtures (wiring) — the modern counterpart to Selenium's ThreadLocal + inheritance patterns.

```
public class SearchPage {
    private final Page page;
    public SearchPage(Page page) { this.page = page; }
    public void search(String q) {
        page.getByRole(AriaRole.TEXTBOX, new
Page.GetByRoleOptions().setName("Search")).fill(q);
        page.getByRole(AriaRole.TEXTBOX, new
Page.GetByRoleOptions().setName("Search")).press("Enter");
    }
}
```

Triage: cross-test pollution → shared context (isolate per test); static Page → inject per test. Anti: static shared Page/Context; inheritance-heavy page hierarchies; Thread.sleep instead of auto-wait. Qs: How do Playwright isolation patterns differ from Selenium ThreadLocal?

31.13 Choosing Patterns Without Overengineering — Refined

Theory: Apply YAGNI (You Aren't Gonna Need It), KISS, Single Responsibility: start with POM + Driver/Request factories + config utility; add DI, Singleton, Strategy, Decorator only when duplication, flakiness, or scaling demands it. Each abstraction must earn its cost in maintenance, onboarding, debug time. Prefer composition, small helpers, TestNG/JUnit lifecycle over custom frameworks; refactor incrementally with metrics (flake rate, onboarding time, change cost). Enterprise Relevance: Over-engineered frameworks (DI containers for 50 tests, 6-layer abstractions) cost more than they save. The senior skill is restraint: simplest thing that scales to the next order of magnitude.

```
// Enough for most suites: factory + POM, no DI container yet
WebDriver driver = DriverFactory.create(System.getProperty("browser", "chrome"));
HomePage home = new HomePage(driver); home.open();
// Add DI when: 3+ implementations, complex lifecycles, cross-cutting concerns
multiply
```

Triage: framework friction (onboarding weeks, debug hours) → remove layers; duplicated code (3rd copy) → extract pattern then, not before. Anti: framework-first (patterns before problems); custom test runner (use TestNG/JUnit); abstraction without duplication evidence. Qs: What earns a new abstraction? What would you remove first?

31.14 Object Mother Pattern for Runtime Variable Data Factory Layouts — Refined

Theory: Object Mother centralizes ready-made test fixtures via static factory methods

(`mothers.validUser()` , `mothers.adminWithExpiredToken()`), hiding complex construction from tests. Complements Builder (Mother names canonical objects; Builder customizes):

`Mothers.admin().toBuilder().role(...)` . Keep mothers small, composable, discoverable; avoid bloated god-mothers with hidden state. Prefer immutable returns + parameters for variations. Provide complete Java code for the runtime generation matrix (per-role/per-state variants). Enterprise Relevance: Mothers give every test one-line canonical data (`validUser()`), eliminating 20-line setup duplication while keeping variations explicit at call site.

```
public final class UserMother {
    private UserMother() {}
    public static User validUser() {
        return User.builder().name("Deloitte SDET").email("sdet-" + UUID.randomUUID() +
"@test.com").role("QA").build();
    }
    public static User admin() { return validUser().toBuilder().role("ADMIN").build(); }
}
    public static User expiredToken() { return
validUser().toBuilder().token("expired").build(); }
}
// test: User u = UserMother.admin(); // canonical + explicit variant
```

Triage: mystery-guest failures (hidden mother default changed) → explicit overrides; god-mother (50 methods) → split by domain. Anti: mutable shared mothers; hidden state; god-mothers; mothers performing I/O (slow). Qs: Mother vs Builder vs Factory? How do you avoid mystery guests?